

# Relazione SIMONE GARAU 20034352

## Progetto implementativo

### 1) Implementazione del Puzzle dell'8 con Algoritmo A\*

Il puzzle dell'8 è un famoso gioco che consiste nel riordinare le tessere numerate in una griglia 3x3, lasciando uno spazio vuoto. Questo spazio consente di spostare le tessere una alla volta al fine di ottenere una configurazione ordinata partendo da una disposizione casuale.

Ho scelto di utilizzare Python per lo sviluppo dell'algoritmo A\* e del resto del codice. Questo algoritmo sfrutta informazioni legate allo stato corrente e stime generate da euristiche.

Durante la valutazione delle possibili configurazioni delle tessere, si crea un grafo in cui ogni nodo rappresenta una configurazione potenziale. Partendo dalla configurazione iniziale della matrice, che è quella casuale, si inizializza un valore a zero, rappresentante il costo che l'algoritmo calcola ad ogni iterazione. L'obiettivo è minimizzare questo costo. Per calcolare il costo, vengono considerati sia il numero di tessere spostate finora, sia un'euristica scelta, come ad esempio la Distanza di Manhattan o il numero di tessere fuori posto. A\* espande i nodi in base al valore del costo, privilegiando quelli con valore minimo, il che permette di esplorare prima i nodi che sembrano promettenti per la soluzione ottimale.

L'utilizzo di un'euristica corretta e ammissibile consente di individuare più rapidamente la soluzione ottimale.

#### Struttura del codice

- **h(state, goal):** Questa funzione calcola la distanza di Manhattan tra la configurazione corrente e quella obiettivo. La distanza di Manhattan è la somma delle distanze orizzontali e verticali tra le posizioni correnti e quelle obiettivo di ogni tessera, escludendo la posizione dello spazio vuoto.
- **h\_out\_of\_place(state, goal):** Questa funzione calcola il numero di tessere fuori posto nella configurazione corrente rispetto a quella obiettivo. Ho implementato anche questa funzione per fare confronti ma nel codice ora viene usata la funzione h().
- **h\_fake(state, goal):** Questa è un'euristica "fittizia" che restituisce sempre 0. È utilizzata per scopi di confronto e valutazione delle prestazioni.
- **getNeighbors(cell, array):** Questa funzione restituisce le posizioni dei "vicini" della cella vuota, cioè le possibili posizioni che potranno diventare cella vuota.
- **reconstructPath(finalNode):** Questa funzione ricostruisce il percorso dalla soluzione trovata fino allo stato iniziale.
- **aStar(initialState, goalState, heuristic=h):** Questa è la funzione principale che implementa l'algoritmo A\*. Utilizza una coda (implementata con un heap) per mantenere traccia dei nodi da esaminare. L'algoritmo continua ad estrarre nodi dalla coda finché non trova la soluzione o esaurisce tutte le possibilità.

## Risultati e Utilizzo del Programma

Una volta definito il codice, viene inizializzata la configurazione iniziale del puzzle e quella obiettivo (è possibile andare manualmente a modificare nel codice la configurazione iniziale definita come un array in *currentState*). Successivamente, l'algoritmo A\* viene eseguito utilizzando l'euristica specificata (in questo caso la distanza di Manhattan) per guidare la ricerca. Se una soluzione viene trovata, viene stampato il percorso dalla configurazione iniziale a quella finale. In caso contrario, viene stampato un messaggio che indica l'impossibilità di trovare una soluzione.

## 2) Implementazione del gioco del Tris con Algoritmo mini-max

Ho sviluppato in linguaggio Python un programma che permette di giocare al gioco del Tris. L'utente può effettuare le sue mosse giocando con il computer, il quale usa l'algoritmo mini-max con un look-ahead di due mosse, insieme a una funzione euristica specificata, per prendere decisioni in modo strategico.

L'algoritmo mini-max è stato implementato per prendere decisioni intelligenti durante il gioco. L'algoritmo esamina tutte le mosse possibili fino a una profondità massima di due mosse, valutando il vantaggio relativo di ciascuna mossa tramite una funzione euristica. In pratica l'algoritmo simula le possibili configurazioni che si possono andare a creare e quando non ci sono più mosse disponibili (e dunque il tabellone è pieno) va ad assegnare un punteggio a quella configurazione per poi confrontarlo successivamente con le altre disposizione che è andato a simulare.

### Struttura del codice

Ho deciso di sviluppare il codice in 2 file: *game.py* che contiene la classe TicTacToe e ospita le funzioni riguardanti il gioco in sé cioè quelle per stampe su terminale o per controllare la situazione nel tabellone; ho creato poi un file *player.py* con le classi Player, HumanPlayer e AIPlayer che contengono le funzioni per la logica del gioco cioè le funzioni per le mosse dell'utente e del computer.

*game.py* contiene:

- le funzioni **make\_board()**, **printBoard()** e **boardNums()** necessarie alla stampa e all'inizializzazione del tabellone.
- **MakeMove()** che esegue la mossa, cioè va a inserire il simbolo corretto nella cella specificata dopo aver controllato che la cella sia libera.
- **winner()** che controlla se c'è un vincitore andando a contare il numero di simboli uguali sulla stessa linea sia in orizzontale, che in verticale, che in diagonale.
- **emptyCells()** è una funzione per la gestione delle celle vuote e controlla se ci sono celle vuote nel tabellone.
- **availableMoves()** restituisce gli indici delle mosse disponibili.

- **is\_game\_over()** serve per controllare se il gioco è terminato andando a vedere se ci sono ancora mosse disponibili o se c'è già un vincitore.
- **play()** è la funzione principale che richiama le funzioni per stampare il tabellone ed eseguire le mosse riferendosi alle classi HumanPlayer ed AIPlayer del file *player.py*.

*player.py* contiene:

- nella classe HumanPlayer la funzione **moveHum()** che prende l'input dell'utente (che inserisce il numero della cella del tabellone) e controlla se quella mossa è ancora disponibile e si può fare; in caso contrario dovrà selezionare un'altra cella.
- nella classe AIPlayer la funzione **moveAI()** che si occupa di fare la mossa per il computer. Se il tabellone è vuoto e quindi si tratta della prima mossa eseguirà una mossa casuale, altrimenti richiama la funzione **minimax()**.
- la funzione **minimax()** che prende in input una configurazione del gioco (state) e il giocatore corrente. Essa esamina tutte le possibili mosse disponibili per il giocatore corrente e valuta il punteggio associato a ciascuna mossa. Quando tutte le celle sono occupate o si raggiunge la profondità massima della ricerca, si raggiunge il caso base e si valuta il punteggio ottenuto tramite la funzione euristica **evaluate()**. Il punteggio migliore ottenuto è quindi restituito come risultato della funzione.
- La funzione **evaluate()** è la funzione che gestisce la valutazione del tabellone e assegna un punteggio a una determinata mossa. In base al tipo di giocatore (se massimizzante o minimizzante) il punteggio viene migliorato di 100, 10 o 1 punto se ci sono rispettivamente 3, 2 o 1 simboli uguali in linea (orizzontale, verticale o diagonale).

## Utilizzo del Programma

Il programma offre una semplice interfaccia da riga di comando che consente agli utenti di giocare contro il computer. Le mosse dei due giocatori si alternano, creando una sfida equilibrata. Il computer utilizza una strategia competitiva grazie alla funzione euristica implementata, rendendo l'esperienza di gioco coinvolgente per gli utenti.

## 3) Implementazione dell'algoritmo Likelyhood weighting

Likelyhood weighting è un algoritmo che permette di stimare le probabilità condizionate di variabili di interesse dato un insieme di evidenze osservate. L'approccio sfrutta il campionamento casuale per generare una serie di campioni compatibili con le evidenze osservate, assegnando loro un peso basato sulla verosimiglianza dei dati osservati.

## Descrizione del Codice Implementato

Il codice Python implementato utilizza la libreria `pysmile`, che fornisce strumenti per la manipolazione e l'analisi delle reti bayesiane. Di seguito è descritto il funzionamento delle principali funzioni:

- **`topological_order(net)`**: Questa funzione implementa l'ordinamento topologico per la rete bayesiana in esame. Definisce un ordine lineare delle variabili nella rete in modo che ogni variabile venga valutata dopo tutte le variabili da cui dipende direttamente.
- **`get_first_index(net, node, probabilities, single_sample)`**: Questa funzione calcola l'indice della prima colonna corrispondente al valore osservato nella tabella delle probabilità condizionate di un nodo.
- **`likelihood_weighting(network, evidence, varX, num_run)`**: Questa è la funzione principale che implementa l'algoritmo Likelihood weighting. Genera un numero specificato di campioni casuali, assegnando loro pesi basati sulla verosimiglianza dei dati osservati. Infine, normalizza i pesi per ottenere le probabilità condizionate desiderate.

## Utilizzo del Programma

Il programma viene utilizzato per stimare le probabilità condizionate di una variabile di interesse, data un'insieme di evidenze osservate in una certa rete bayesiana. Viene specificato il file della rete bayesiana, le evidenze osservate, la variabile di interesse e il numero di campioni da generare. Il risultato restituito è la probabilità condizionata stimata per la variabile di interesse, data l'evidenza osservata.

Il codice, in questo caso, legge la rete contenuta nel file `"WetGrass.xdsl"` ed esegue quindi l'algoritmo Likelihood weighting per stimare la probabilità condizionata della variabile `"Rain"` data l'evidenza che lo `"Sprinkler"` è acceso.

## **Progetto supporto alle decisioni**

### **1) Problema Industria manifatturiera e lettura rete**

Il problema riguardava la decisione da parte di un'industria manifatturiera di proseguire con la produzione di un nuovo prodotto o fermarla. La decisione sarebbe stata presa basandosi sulla possibilità di eseguire ricerche di mercato o migliorare la qualità del prodotto.

Per la risoluzione di questo problema ho costruito un modello sfruttando il software Genie. Ho creato nodi probabilistici che mi hanno permesso di andare a inserire le probabilità specificate nel testo dove possibile (nel caso ad esempio del nodo *Domanda effettiva* non ho modificato la probabilità di default poiché nel testo non vi erano informazioni a riguardo). Ho creato 2 nodi decisione per permettere all'utente di andare a effettuare le scelte di eseguire ricerche di mercato o migliorare la qualità del prodotto. Siccome ogni decisione ha un costo ho creato 2 nodi utilità per andare a esprimere il costo delle possibili scelte (*Costo\_ricerca* e *Costo miglioramento*); esiste inoltre un terzo nodo utilità che valuta il profitto. Questi ultimi 3 nodi confluiscono poi in un quarto nodo utilità chiamato *Utilità finale* che esegue la somma dei nodi utilità.

### **Descrizione del Codice per la lettura della rete Bayesiana**

Sfruttando le API fornite da Smile è stato possibile sviluppare un programma Python che permette di leggere la rete creata con Genie ed eseguire operazioni su di essa.

Al lancio del programma ci viene subito mostrato l'elenco dei nodi presenti nella rete. Successivamente l'utente può subito fare la scelta se eseguire o meno la ricerca di mercato: viene specificato il costo della ricerca ed è possibile inserire la risposta da tastiera. Allo stesso modo, sia in caso di risposta affermativa che negativa, l'utente potrà ancora scegliere se migliorare la qualità del prodotto e a quale costo. Dopo quest'ultima scelta verrà mostrato a schermo il valore del profitto calcolato in base ai dati del problema e infine sarà possibile visualizzare il valore dell'utilità finale influenzata dal profitto dell'azienda e dai costi sostenuti per le decisioni prese precedentemente.

### **2) Problema veicolo autonomo e lettura rete**

Il testo del problema richiedeva di modellare una rete bayesiana per la gestione del comportamento di un veicolo autonomo. Il veicolo può spostarsi in 3 direzioni (destra, sinistra o rimanere fermo) e l'obiettivo è di rimanere nella corsia centrale. Il veicolo dispone di un sensore che con una certa probabilità rileva la sua posizione. L'accuratezza del sensore è influenzata dallo stato del terreno su cui si muove il veicolo e dalle condizioni del tempo. Il problema richiedeva anche di modellare il processo in 5 istanti temporali.

Ho costruito un modello sul software Genie inserendo i nodi e collegandoli tra loro in base al loro rapporto di dipendenza. Dopo aver inserito i nodi opportuni nel Temporal plate e nel Term Conditions ho eseguito l'unroll della mia rete per poi scrivere il codice Python che la interrogasse. Ho creato subito i nodi *Meteo* e *Stato terreno* evitando di specificare le probabilità poiché non fornite nel testo. In seguito ho creato un nodo NoisyMax chiamato *SensoreGuasto* che mi ha

permesso di gestire meglio l'incertezza e di inserire le probabilità del meteo umido fornite nel testo. Anche il nodo *AccuratezzaSensore* l'ho modellato come un nodo NoisyMax per meglio gestire le probabilità di tempo umido e terreno sconnesso. In seguito ho creato un nodo *PosizioneRilevata*, che è influenzato dal nodo *PosizioneReale*, e ho inserito le corrette probabilità fornite nel testo. Dopo ho creato il nodo decisione che permette di effettuare la scelta su quale direzione il veicolo dovrà prendere. Infine ho creato il nodo *posizioneFinale* e l'ho inserito nella Term conditions: questo nodo riguarda la posizione del veicolo dopo i 5 istanti temporali e di conseguenza la rispettiva utilità finale definita in base alla posizione definitiva che ha raggiunto il veicolo.

### **Descrizione del Codice per la lettura della rete Bayesiana**

Ho sviluppato un programma Python che legge la rete che ho creato su Genie e permette all'utente di interagire con essa (il programma legge il modello unrolled). L'utente dopo aver lanciato il programma, può interagire da riga di comando, dove vengono inizialmente visualizzate le probabilità con cui il veicolo si trova nella corsia di sinistra, di destra o al centro. Dopo aver settato l'evidenza ci sarà chiesto che azione dovrà fare il veicolo: l'utente, seguendo le istruzioni, potrà indicare la decisione e in seguito verranno calcolate le probabilità di posizione del veicolo in base al sensore di posizione. Raggiunto il secondo istante temporale l'utente dovrà di nuovo inserire l'evidenza in base alle probabilità che il sensore di posizione ha calcolato e dopo selezionare l'azione da effettuare. Questi passaggi verranno eseguiti fino all'ultimo istante di tempo dove verranno mostrate le probabilità con cui il veicolo si trovi a sinistra, destra o al centro in base alle scelte fatte in precedenza. Alla fine verrà visualizzato il valore dell'utilità finale calcolata rispetto all'obiettivo del problema.